

Component Reference — Snoopy HQ Journal

Landed here first? This is a per-file code reference, useful once you already know the project. If it is new to you, read [GETTING-STARTED.md](#) , then [PROJECT-HANDBOOK.md](#) , and come back here when you need a specific component. All documents are routed from the [documentation index](#).

Every component in the codebase, what it renders, what it owns, and how it was implemented.

Companion to [PROJECT-HANDBOOK.md](#) (architecture, content pipeline, build) and [MAINTAINER-MANUAL.md](#) (day-to-day content tasks). This document is the code-level one: it assumes you can read TypeScript and React but knows nothing about *this* repo.

Contents

- 0. How to read this document
 - 1. Render trees
 - 2. The client/server split at a glance
 - 3. Shared primitives — [BrandLockup](#) , [SectionHeading](#) , [EditorialArt](#) , [ResponsiveImage](#)
 - 4. [Page chrome](#) — header, theme, search, footer, newsletter
 - 5. [Home page sections](#) — featured, grids, cards, categories
 - 6. [The article page](#) — view, header, body, tags, table of contents
 - 7. [Images](#) — article photo, zoom dialog, photo permalink
 - 8. [Documents](#) — the eleven-file preview pipeline
 - 9. [Motion infrastructure](#) — reveal, press, page transition, scroll memory
 - 10. [Layout and theme bootstrap](#)
 - 11. [Files that are not part of the app](#)
 - 12. [Recurring patterns](#)
 - 13. [Where to make a given change](#)
-

o. How to read this document

Each entry follows the same shape:

Field	Meaning
File	Path from repo root.
Kind	Server = React Server Component (no <code>"use client"</code> ; runs only on the Worker, ships zero JS). Client = has <code>"use client"</code> ; hydrates in the browser. Module = plain <code>.ts</code> , no JSX.
Rendered by	Who mounts it.
Props	Full prop list with defaults.
State / refs	Everything the component owns across renders.
How it works	The actual mechanics, step by step.
CSS hooks	Class names and data attributes <code>app/globals.css</code> targets. Change one and the styling breaks.
Accessibility	ARIA, focus, keyboard, live regions.

A rule that holds across the whole tree: **the default is Server**. `"use client"` appears only where the component needs state, a DOM API, or an event handler. `app/components/` holds 45 files; exactly 20 carry `"use client"` . The other 25 — components, the barrel export, and four plain modules — render once on the Worker and ship as HTML. (One more client file, `components/ui/text-reveal.tsx` , lives outside `app/` and is unreferenced; see §11.)

1. Render trees

Home — app/page.tsx

```
RootLayout (Server)
└─ PageTransition (Client)          ← wraps every route
    └─ BlogHeader (Client)
        │   └─ ScrollAwareHeader (Client) ← <header data-condensed>
        │       │   └─ BrandLockup (Server)
        │       │   └─ nav (inline)
        │       │   └─ ThemeToggle (Client)
        │       └─ SearchDialog (Client)
        └─ main#main-content
            │   └─ JournalIntro (Server)
            │   └─ MotionReveal → FeaturedStories (Server)
            │       │   └─ SectionHeading (Server)
            │       │   └─ MotionCard (Client) → Link → ArticleCardMedia (Server)
            │       │       └─ ResponsiveImage | EditorialArt
            │   └─ MotionReveal → LatestStories (Server)
            │       │   └─ PostCard (Server) ×N → MotionCard → ArticleCardMedia
            │   └─ MotionReveal → CategoryBrowser (Client)
            │   └─ MotionReveal → NewsletterSignup (Server) → NewsletterForm (Client)
            └─ JournalFooter (Server) → BrandLockup
```

Article — app/blog/[slug]/page.tsx

```
BlogHeader
main#main-content
└─ ArticleView (Server)
    │   └─ nav.post-breadcrumb
    │   └─ ArticleHeader (Server) → ShareButton (Client)
    │   └─ ArticlePhoto (Client, hero) | EditorialArt (Server, fallback)
    │   └─ div.article-reading-layout
    │       │   └─ ArticleTableOfContents (Client) ← only if 4+ sections or 1000+ words
    │       │   └─ ArticleBody (Server)
    │       │       │   └─ MotionReveal → section
    │       │       │       │   └─ ArticlePhoto (Client) ← placed by afterParagraph
    │       │       │       │   └─ ArticleDocument (Client) ← placed by afterParagraph
    │       │       └─ ArticleTags (Server)
    └─ MotionReveal → RelatedStories (Server)
    └─ MotionReveal → NewsletterSignup variant="article"
JournalFooter
```

Photo focus — `app/photo/[id]/page.tsx`

```
PhotoFocusView (Server)      ← no BlogHeader; its own minimal toolbar
├─ ArticleReturnLink (Client) ×2 ← writes scroll-restore intent
└─ ResponsiveImage (Server)
```

Note this route renders no site header and no footer. It is a deliberately bare, full-bleed page for one photograph.

2. The client/server split at a glance

Client component	Why it must be client
<code>BlogHeader</code>	menu + search open state, Escape key handling
<code>ScrollAwareHeader</code>	reads <code>window.scrollTo</code>
<code>ThemeToggle</code>	reads/writes <code>document.documentElement.dataset.theme</code>
<code>SearchDialog</code>	input state, focus trap, body scroll lock
<code>NewsletterForm</code>	form submit state
<code>CategoryBrowser</code>	accordion open state
<code>MotionCard</code> , <code>MotionReveal</code> , <code>PageTransition</code>	framer-motion needs hooks
<code>ShareButton</code>	<code>navigator.share</code> / clipboard
<code>ArticlePhoto</code>	opens the focused-image dialog
<code>FocusedImageDialog</code> , <code>ImageZoomControls</code>	pointer/wheel/keyboard zoom
<code>ArticleTableOfContents</code>	scroll-spy
<code>ArticleReturnLink</code>	writes <code>sessionStorage</code> on click
<code>ArticleDocument</code> , <code>DocumentPreviewDialog</code> , <code>DocumentPreviewWorkspace</code> , <code>DocumentViewer</code> , <code>DocumentViewerErrorBoundary</code>	dynamic import + browser-only parsers

That is 20 files. The remaining 25 in `app/components/` are server-rendered and ship no JavaScript.

3. Shared primitives

BrandLookup

File `app/components/brand-lookup.tsx` · **Kind** Server · **Rendered by** `BlogHeader`, `JournalFooter`

Prop	Type	Default
<code>href</code>	<code>string</code>	<code>"/"</code>
<code>ariaLabel</code>	<code>string</code>	<code>"Snoopy HQ Journal home"</code>

The wordmark, used twice on every page. The logo glyph is not an image — it is two nested spans that CSS shapes into the mark:

```
<span className="brand-mark" aria-hidden="true"><span>S</span><i /></span>
<span><strong>Snoopy HQ</strong><small>Journal</small></span>
```

`aria-hidden` on the mark means a screen reader announces only the accessible name from `aria-label`, not a stray "S".

CSS hooks `.brand`, `.brand-mark`

SectionHeading

File `app/components/section-heading.tsx` · **Kind** Server

Prop	Type	Notes
<code>eyebrow</code>	<code>string</code>	small caps label above the title
<code>title</code>	<code>string</code>	rendered as <code><h2></code>
<code>titleId</code>	<code>string?</code>	id so the parent <code><section></code> can point <code>aria-labelledby</code> at it
<code>description</code>	<code>string?</code>	optional paragraph
<code>action</code>	<code>ReactNode?</code>	optional right-hand slot (used by <code>RelatedStories</code> for its "View all" link)

The `titleId` prop is the important part of the design. Every section on the home page is an `aria-labelledby` region, and the label lives inside this component, so the id has to be passed in rather than generated:

```
<section className="latest-section" id={sectionId} aria-labelledby={` ${sectionId}-title`} >
  <SectionHeading titleId={` ${sectionId}-title`} ... />
```

`action` is a `ReactNode` rather than a `{label, href}` pair — the component does not need to know what the action is, so it does not model it.

CSS hooks `.editorial-heading`, `.eyebrow`

EditorialArt

File `app/components/editorial-art.tsx` · **Kind** Server

Prop	Type	Default
<code>label</code>	<code>string</code>	—
<code>variant</code>	<code>ArtVariant</code>	—
<code>compact</code>	<code>boolean</code>	<code>false</code>
<code>caption</code>	<code>string</code>	<code>"GOOD\nGRIEF!"</code>

The fallback artwork for any article with no photograph. Pure CSS illustration — sun, doghouse, horizon line — built from empty divs; there is no image file involved:

```
<div className={`editorial-art scene-${variant}${compact ? " compact" : ""}`} aria-  
hidden="true">  
  <span className="art-number">{label}</span>  
  <div className="art-sun" />  
  <div className="art-house"><i /></div>  
  <div className="art-line" />  
  <strong>{caption.split("\n").map((line, index) => <span key={` ${line}-${index}`}>{line}  
</span>)}</strong>  
</div>
```

`variant` selects a palette/composition via the `scene-*` class (`ArtVariant` is a union type defined in `app/content/types.ts`). The whole block is `aria-hidden` — it is decoration and carries no information a reader would miss.

The file also re-exports the type so consumers importing art can pull the type from the same module:

```
export type { ArtVariant } from "../content";
```

CSS hooks `.editorial-art`, `.scene-{variant}`, `.compact`, `.art-sun`, `.art-house`, `.art-line`, `.art-number`

ResponsiveImage

File `app/components/media/responsive-image.tsx` · **Kind** Server

The single point where an image asset becomes HTML. Nothing else in the codebase writes an `<img>` for content imagery.

Prop	Type	Default
image	MediaAsset	—
className	string?	applied to <picture>
loading	"eager" "lazy"	"lazy"
sizes	string?	see below
fullResolution	boolean	false

How it works

1. `optimizedPath` rewrites the source path into the pre-generated variant path produced by `scripts/optimize-images.mjs`:

```
function optimizedPath(src: string, width: number, format: "avif" | "webp") {
  const extensionIndex = src.lastIndexOf(".");
  const basename = extensionIndex === -1 ? src : src.slice(0, extensionIndex);
  return `/_optimized${basename}-${width}.${format}`;
}
```

`/images/articles/art_x/asset_y.jpg` → `/_optimized/images/articles/art_x/asset_y-768.avif`.

2. `sourceSet` builds the candidate list. It keeps only widths **smaller** than the real image, then appends the true width — so a 900px-wide photo never advertises a 1600px variant that was never generated:

```
const widths = [...responsiveWidths.filter((width) => width < image.width), image.width];
```

`responsiveWidths` is `[480, 768, 1200, 1600]`, matching the ladder the optimizer script emits.

3. Three-tier fallback: `<source type="image/avif">`, then `<source type="image/webp">`, then a plain `<img src={image.src}>` pointing at the untouched original. If the optimizer never ran, the AVIF and WebP requests 404 and the browser falls back to the original — the page degrades, it does not break.
4. `sizes` resolution order: `fullResolution` (used by the zoom dialog, forces `${image.width}px`) → explicit `sizes` prop → a default keyed off `image.portrait`.
5. `width / height` are always emitted from the asset record, which is what prevents layout shift. `fetchPriority="high"` is set only when `loading="eager"`.

Why not `next/image`? The optimizer is a build step that writes static files, and the app runs as a Cloudflare Worker with no image-optimization runtime. A plain `<picture>` needs no server at request time.

4. Page chrome

`BlogHeader`

File `app/components/blog-header.tsx` · **Kind** Client · **Props** none

The site header. Owns two independent booleans and delegates the visual behaviour to `ScrollAwareHeader`.

State / refs

Name	Purpose
<code>menuOpen</code>	mobile nav open
<code>searchOpen</code>	search overlay open
<code>closeSearch</code>	<code>useCallback</code> — stable identity so <code>SearchDialog</code> 's effect does not re-run every render
<code>menuButtonRef</code>	focus target on Escape
<code>navigationRef</code>	used to find the first link to focus

How it works

The navigation model is a frozen array outside the component, so it is allocated once:

```
const navigation = [  
  { href: "/#latest", label: "Latest" },  
  { href: "/#archive", label: "Archive" },  
  { href: "/#topics", label: "Topics" },  
];
```

Opening the menu triggers one effect that does two jobs — move focus into the menu, and register an Escape handler that closes it and returns focus to the button:

```
useEffect(() => {
  if (!menuOpen) return;
  navigationRef.current?.querySelector<HTMLAnchorElement>("a")?.focus();

  const closeOnEscape = (event: KeyboardEvent) => {
    if (event.key !== "Escape") return;
    setMenuOpen(false);
    menuButtonRef.current?.focus();
  };

  window.addEventListener("keydown", closeOnEscape);
  return () => window.removeEventListener("keydown", closeOnEscape);
}, [menuOpen]);
```

The `if (!menuOpen) return` guard is the whole subscription lifecycle: the listener exists only while the menu is open.

Accessibility

- First child is the skip link: `<a className="skip-link" href="#main-content">`. Every page renders `<main id="main-content">` for it to land on.
- Menu button carries `aria-controls="main-navigation"` and `aria-expanded`, and its `aria-label` flips between "Open navigation" and "Close navigation".
- Every nav link closes the menu on click, so tapping a hash link on mobile does not leave the panel covering the destination.

CSS hooks `.blog-header-inner`, `.main-nav`, `.main-nav.open`, `.header-tools`, `.icon-button`, `.menu-button`, `.subscribe-link`, `.skip-link`

ScrollAwareHeader

File `app/components/navigation/scroll-aware-header.tsx` · **Kind** Client

Prop	Type
<code>children</code>	<code>ReactNode</code>

Renders `<header className="blog-header" data-condensed="true|false">` and nothing else. All the shrinking is CSS reacting to that attribute.

How it works — three details worth understanding

1. **Hysteresis.** Two thresholds, not one:

```
const CONDENSE_AT = 72;
const EXPAND_AT = 16;

const next = condensedRef.current
  ? window.scrollY > EXPAND_AT    // already condensed: stay condensed until well above
    the top
  : window.scrollY >= CONDENSE_AT; // expanded: don't condense until clearly scrolled
```

With a single threshold, a scroll position sitting exactly on the boundary makes the header flicker between states as the layout height changes. The 56px dead band removes that feedback loop.

2. **rAF coalescing.** Scroll fires far more often than the screen repaints, so the handler only schedules:

```
const scheduleUpdate = () => {
  if (animationFrame) return;
  animationFrame = window.requestAnimationFrame(update);
};
window.addEventListener("scroll", scheduleUpdate, { passive: true });
```

`{ passive: true }` tells the browser the handler will never call `preventDefault`, so scrolling is never blocked on it.

3. **A ref shadows the state.** `condensedRef` holds the same value as `condensed` because `update` runs outside React's render cycle and needs the current value without re-subscribing. `setCondensed` is called only when the value actually changes, so a scroll from 0 to 5000px causes exactly one re-render.

`pageshow` is also subscribed: returning to the page via the back/forward cache restores scroll position without firing `scroll`, and without this the header state would be stale.

ThemeToggle

File `app/components/theme-toggle.tsx` · **Kind** Client · **Props** none

Light/dark switch. Notable for what it does *not* do: there is no React state and no context provider.

How it works

The source of truth is the DOM attribute `document.documentElement.dataset.theme`. The component subscribes to it with `useSyncExternalStore`:

```
function subscribeToTheme(onChange: () => void) {
  window.addEventListener("snoopy-theme-change", onChange);
  return () => window.removeEventListener("snoopy-theme-change", onChange);
}

function currentThemeIsDark() {
  return document.documentElement.dataset.theme === "dark";
}

const dark = useSyncExternalStore(subscribeToTheme, currentThemeIsDark, () => false);
```

The three arguments are: subscribe, read-on-client, read-on-server. The server snapshot returns `false` so SSR always renders the light-mode label; the correct value is picked up on hydration.

Toggling writes four things and then broadcasts:

```
document.documentElement.dataset.theme = nextDark ? "dark" : "light";
document.documentElement.style.colorScheme = nextDark ? "dark" : "light";
localStorage.setItem("snoopy-theme", nextDark ? "dark" : "light");
window.dispatchEvent(new Event("snoopy-theme-change"));
```

- `dataset.theme` drives the CSS custom properties.
- `style.colorScheme` tells the browser to restyle native UI (scrollbars, form controls, the flash between pages).
- `localStorage` persists it for the inline boot script in `app/layout.tsx`.
- The custom event is what lets *any* number of subscribers stay in sync — `DocumentPreviewDialog` watches the same attribute via `MutationObserver` and re-themes the file viewer.

Both icons are always rendered; CSS shows one. That avoids a layout jump on toggle.

Accessibility `aria-pressed={dark}` marks it as a toggle button, and the label states the destination ("Switch to dark theme"), not the current state.

SearchDialog

File `app/components/search-dialog.tsx` · **Kind** Client

Prop	Type
<code>open</code>	<code>boolean</code>
<code>onClose</code>	<code>() => void</code>

Client-side full-text search over every post. There is no search API — `posts` is imported directly and the whole corpus ships inside the client bundle, which is viable because the site has tens of articles, not thousands.

Matching

```
const results = useMemo(() => {
  const needle = query.trim().toLowerCase();
  if (!needle) return posts.slice(0, 4);
  return posts.filter((post) =>
    `${post.title} ${post.summary} ${post.category} ${post.tags.join(" ")}`
    .toLowerCase().includes(needle));
}, [query]);
```

Four fields are concatenated into one haystack per post; the match is a case-insensitive substring. Empty query shows the four most recent posts rather than nothing, so the panel is never blank.

The modal effect — one `useEffect` gated on `open` handles the full modal contract:

1. Remember `document.activeElement` so focus can be restored on close.
2. Save and overwrite `document.body.style.overflow` to lock background scroll (saving the previous value matters — hard-coding `""` on cleanup would clobber a lock owned by another dialog).
3. Focus the input.
4. Install a keydown handler for Escape and Tab.
5. Cleanup restores overflow, removes the listener, and refocuses the previous element.

The focus trap — recomputed on every Tab so it stays correct as results change:

```
const focusable = Array.from(dialogRef.current.querySelectorAll<HTMLElement>(
  "button, input, a[href], [tabindex]:not([tabindex='-1'])"));
const first = focusable[0];
const last = focusable[focusable.length - 1];

if (event.shiftKey && document.activeElement === first) {
  event.preventDefault();
  last.focus();
} else if (!event.shiftKey && document.activeElement === last) {
  event.preventDefault();
  first.focus();
}
```

Motion — `AnimatePresence initial={false}` (no animation on first paint), a 160ms overlay fade, and a 220ms panel slide on the `[0.22, 1, 0.36, 1]` ease-out curve. Every value collapses to `0` when `useReducedMotion()` is true, and `initial` becomes `false` so the element mounts in its final state rather than animating from a hidden one.

Backdrop dismiss uses `onMouseDown` with an identity check:

```
onMouseDown={(event) => event.target === event.currentTarget && onClose()}
```

`event.target === event.currentTarget` means the press landed on the overlay itself, not a child. Using `mousedown` rather than `click` avoids the case where a drag that starts inside the panel and ends on the backdrop closes the dialog.

Accessibility `role="dialog"`, `aria-modal="true"`, `aria-labelledby` pointing at the `<h2>`, `.sr-only` label on the input, and a spoken empty state ("No articles found. Try a broader word.").

JournalIntro

File `app/components/journal-intro.tsx` · **Kind** Server · **Props** none

The hero. Static markup — the only `<h1>` on the home page, plus a jump link to `#latest`. The `<br />` in the headline is intentional typographic control over the line break, not a layout hack.

JournalFooter

File `app/components/journal-footer.tsx` · **Kind** Server · **Props** none

Brand lockup, one line of copy, three links inside a `<nav aria-label="Footer navigation">`. The label matters: a page with two `<nav>` elements needs both distinguished for screen-reader landmark navigation (the header's is `aria-label="Main navigation"`).

NewsletterSignup

File `app/components/newsletter-signup.tsx` · **Kind** Server

Prop	Type	Default
<code>variant</code>	<code>"home" "article"</code>	<code>"home"</code>

Two layouts around the same form. `"home"` renders a decorative stamp block (`.newsletter-art`, `aria-hidden`); `"article"` is a plainer band. Implemented as an early return rather than conditional class names, because the two variants have genuinely different markup:

```
if (variant === "article") {
  return <section className="article-newsletter" id="newsletter">...</section>;
}
return <section className="newsletter" id="newsletter">...</section>;
```

Both carry `id="newsletter"` — the "Subscribe" link in the header targets it on whichever page you are on.

NewsletterForm

File `app/components/newsletter-form.tsx` · **Kind** Client · **Props** none

This form does not submit anywhere. It exists so the page is complete; wiring it to a provider is a documented open item in `PROJECT-HANDBOOK.md` §21.

```
const submit = (event: FormEvent) => { event.preventDefault(); setJoined(true); };

if (joined) {
  return <p className="newsletter-success" role="status">
    <Check size={18} /> Thanks – this preview did not store your email.
  </p>;
}
```

The success copy tells the truth rather than faking a subscription. `role="status"` makes the swap audible to a screen reader. Validation is native: `required type="email"` plus `autoComplete="email"`.

5. Home page sections

FeaturedStories

File `app/components/featured-stories.tsx` · Kind Server

Prop	Type
<code>lead</code>	<code>BlogPost</code>
<code>secondary</code>	<code>readonly BlogPost[]</code>

The top-of-page trio: one large story plus two smaller. Fed from `featuredPosts` (which is `posts.slice(0, 3)`):

```
<FeaturedStories lead={featuredPosts[0]} secondary={featuredPosts.slice(1)} />
```

The lead image is the page's Largest Contentful Paint element, so it is the only image on the home page marked `priority` — which becomes `loading="eager"` and `fetchPriority="high"` inside `ResponsiveImage`.

Secondary cards get generated labels `02`, `03`:

```
<ArticleCardMedia post={post} label={`0${index + 2}`} compact />
```

Per-post accent colouring is a class, not an inline style: `className={`lead-story accent-${lead.accent}`}`. The accent is one of a fixed union in the content type, so the CSS for every possible value exists.

LatestStories

File `app/components/latest-stories.tsx` · Kind Server

Prop	Type	Default
<code>posts</code>	<code>readonly BlogPost[]</code>	—
<code>startIndex</code>	<code>number</code>	<code>0</code>
<code>sectionId</code>	<code>string</code>	<code>"latest"</code>
<code>eyebrow</code>	<code>string</code>	<code>"The latest"</code>
<code>title</code>	<code>string</code>	<code>"Stories, guides & good grief"</code>
<code>description</code>	<code>string</code>	(long default)

A configurable grid section. Everything but `posts` has a default, so the common case is `<LatestStories posts={...} />`; the home page overrides all of them to render the archive band:


```

<LatestStories
  posts={archivePosts}
  sectionId="archive"
  eyebrow="From the archive"
  title="More stories for slower moments"
  description="The rest of the journal, ordered from newest to oldest..."
/>

```

`sectionId` does double duty: it is the anchor target *and* the prefix for the heading id used by `aria-labelledby`, which keeps the two in sync by construction.

Card labels are zero-padded and offset by `startIndex`:

```
label={`A${`${index + startIndex + 1}`.padStart(2, "0")}`} // A01, A02, ...
```

PostCard

File `app/components/post-card.tsx` · **Kind** Server

Prop	Type	Default
<code>post</code>	<code>BlogPost</code>	—
<code>label</code>	<code>string</code>	—
<code>variant</code>	<code>ArtVariant</code>	<code>post.art</code>
<code>showDate</code>	<code>boolean</code>	<code>true</code>
<code>showAction</code>	<code>boolean</code>	<code>true</code>

The standard grid card, used by `LatestStories` and `RelatedStories`.

The **whole card is one** `<a>` — media, meta, heading, summary, and the "Read article" affordance all live inside a single `<Link>`. This is deliberate: one link per card means one tab stop and one large hit target, instead of a screen-reader user hearing three links to the same URL. The "Read article" text is a `<span>`, not a nested anchor.

`showDate` and `showAction` exist because `RelatedStories` renders a denser variant:

```
<PostCard post={post} label={`0${index + 1}`} showDate={false} showAction={false} />
```

The card is wrapped in `MotionCard` for the press effect. Note the ordering — `MotionCard` (client) is the outer element and the link is its child, so the server-rendered card content passes through as `children` and is never pulled into the client bundle.

ArticleCardMedia

File `app/components/article-card-media.tsx` · **Kind** Server

Prop	Type	Default
<code>post</code>	<code>BlogPost</code>	—
<code>label</code>	<code>string</code>	—
<code>compact</code>	<code>boolean</code>	<code>false</code>
<code>priority</code>	<code>boolean</code>	<code>false</code>
<code>fallbackVariant</code>	<code>ArtVariant</code>	<code>post.art</code>

Decides what a card shows above the title: a real photograph or generated art.

```
const image = mediaForArticle(post.id)[0];

if (!image) {
  return <EditorialArt label={label} variant={fallbackVariant} caption={post.artLabel} compact={compact} />;
}
```

`mediaForArticle(post.id)` reads the media registry and returns that article's images in order; index `0` is the de-facto cover. There is no "cover" flag in the content model — the first image wins.

When a photo exists, the component picks the `sizes` string, and this is where the responsive contract for cards is defined:

```
sizes={compact
  ? "(max-width: 640px) calc(100vw - 68px), (max-width: 900px) calc(50vw - 44px), 340px"
  : "(max-width: 900px) calc(100vw - 40px), 720px"}
```

Those `calc()` terms subtract the real page gutters and grid gaps, so the browser downloads a variant matched to the rendered box rather than the viewport.

`data-media-asset-id={image.id}` is emitted on the wrapper — a debugging affordance that lets you map any card on screen back to a record in `content/assets/`.

CategoryBrowser

File `app/components/category-browser.tsx` · **Kind** Client

Prop	Type
<code>categories</code>	<code>readonly CategoryDefinition[]</code>
<code>posts</code>	<code>readonly BlogPost[]</code>

Accordion of topics. Single-open by design — the state is the open category *name*, not a set:

```
const [openCategory, setOpenCategory] = useState(categories[0]?.name ?? "");
...
onClick={() => setOpenCategory(isOpen ? "" : category.name)}
```

Clicking the open one sets "", closing everything. The first category is open on mount so the section is never a wall of collapsed bars.

Filtering happens per render, inline:

```
const categoryPosts = posts.filter((post) => post.category === category.name);
```

No memo. With a handful of categories over a few dozen posts this is a few hundred string comparisons — cheaper than the bookkeeping to avoid it.

Accessibility — the accordion follows the standard disclosure pattern:

- The trigger is a `<button>` *inside* an `<h3>`, so the heading stays in the document outline and the control stays operable.
- `aria-expanded={isOpen}` and `aria-controls={panelId}` on the button; `id={panelId}` on the panel.
- The panel uses the real `hidden` attribute (`hidden={!isOpen}`), not `display: none` from a class — hidden content is removed from the accessibility tree and from tab order without CSS having to cooperate.
- Count text pluralises: `{n} {n === 1 ? "story" : "stories"}`.

CSS hooks `.category-item`, `.is-open`, `.category-panel`, `.category-accordion`

RelatedStories

File `app/components/related-stories.tsx` · **Kind** Server

Prop	Type
<code>posts</code>	<code>readonly BlogPost[]</code>

The "keep reading" band at the bottom of an article. The related list is resolved in the page, not here — `app/blog/[slug]/page.tsx` maps each reference through both lookup functions, so the `related` field in content may hold ids *or* slugs:

```
const related = post.related
  .map((reference) => postById(reference) ?? postBySlug(reference))
  .filter((item) => item !== undefined);
```

A reference that resolves to nothing is dropped silently — a deleted article cannot break the pages that pointed at it.

6. The article page

ArticleView

File app/components/article-view.tsx · **Kind** Server

Prop	Type
post	BlogPost

Composes the whole article and makes exactly one decision: whether the piece is long enough to deserve a table of contents.

```
const wordCount = post.sections.reduce(
  (total, section) => total + section.paragraphs.join(" ").split(/\s+/).filter(Boolean).length,
  0,
);
const hasTableOfContents = post.sections.length >= 4 || wordCount >= 1000;
```

Two triggers, either sufficient: a piece with many short sections benefits from navigation, and so does a long piece with few. `.filter(Boolean)` drops the empty strings that `split(/\s+/)` produces at string edges.

The result also drives the layout class, because the reading grid changes shape when a sidebar exists:

```
<div className={`article-reading-layout${hasTableOfContents ? " has-toc" : ""}`>
```

Hero selection is the same fallback rule as cards, one level up:

```
{post.heroImage
  ? <ArticlePhoto image={post.heroImage} hero priority />
  : <EditorialArt label="SNOOPY HQ / JOURNAL" variant={post.art} caption={post.artLabel} />}
```

ArticleHeader

File app/components/article-header.tsx · **Kind** Server

Prop	Type
post	BlogPost

Category eyebrow, `<h1>`, summary, byline, share button.

Content stores dates as human strings ("22 July 2026"), which is what should be *displayed*. Machines need ISO, so the component converts:

```
const months = ["January", "February", ..., "December"];

function machineDate(date: string) {
  const [day, month, year] = date.split(" ");
  const monthNumber = months.indexOf(month) + 1;
  return `${year}-${`${monthNumber}`.padStart(2, "0")}-${day.padStart(2, "0")}`;
}
```

Rendered as `<time dateTime={machineDate(post.date)}>{post.date}</time>` — readable text, parseable attribute.

Two details that look odd and are not:

- `<h1 tabIndex={-1}>` — not focusable by Tab, but focusable by script. The table of contents' "Back to the beginning" link calls `heading?.focus({ preventScroll: true })` so keyboard and screen-reader users land *at the heading*, not merely at that scroll offset.
- `id="article-start"` on the `<header>` is the target that link scrolls to.

ShareButton

File `app/components/share-button.tsx` · **Kind** Client · **Rendered by** `ArticleHeader`

Prop	Type
<code>title</code>	<code>string</code>

Progressive enhancement over two browser APIs:

```
if (navigator.share) {
  await navigator.share({ title, url: window.location.href });
  return;
}

await navigator.clipboard.writeText(window.location.href);
setCopied(true);
window.setTimeout(() => setCopied(false), 1600);
```

Native share sheet where it exists (mobile, Safari), clipboard everywhere else. The "Copied" confirmation reverts after 1.6s.

The error branch is the interesting part. Dismissing the OS share sheet **rejects** the promise, and that is not a failure:

```
const wasCanceled = error instanceof Error && (
  error.name === "AbortError" || error.message.toLowerCase().includes("cancel")
);

if (!wasCanceled) {
  setErrorMessage("Sharing was unavailable. Copy the address from your browser instead.");
}
```

Only a genuine failure produces a message. Both outcomes are announced through one polite live region:

```
<span className="sr-only" role="status" aria-live="polite">
  {copied ? "Article link copied." : errorMessage}
</span>
```

The click handler is `onClick={() => void share()}` — the `void` explicitly discards the promise, which is how you tell both TypeScript and the next reader that the rejection is handled inside.

ArticleTags

File `app/components/article-tags.tsx` · **Kind** Server

Prop	Type
<code>tags</code>	<code>readonly string[]</code>

Renders "Filed under" plus a `<ul>`. Returns `null` for an empty list rather than an empty heading. The tags are **not links** — there are no tag archive pages, and rendering dead links would be worse than rendering text.

ArticleBody

File `app/components/article-body.tsx` · **Kind** Server

Prop	Type
<code>post</code>	<code>BlogPost</code>

The layout engine. Turns the content object into the article DOM, and implements the `afterParagraph` placement rule that the whole authoring model rests on.

Inline emphasis — the only inline markup the renderer supports:

```
function renderInlineEmphasis(text: string) {
  return text.split(/(\*[^\*]+\*)/g).map((part, index) => (
    part.startsWith("*") && part.endsWith("*")
      ? <em key={index}>{part.slice(1, -1)}</em>
      : part
  ));
}
```

The capturing group in the split keeps the delimiters in the output array, so alternating plain/emphasised parts survive. No bold, no links, no HTML — because content is data, and letting arbitrary markup through would put an injection surface into every article. Anything richer belongs in the `references` or `list` fields.

Placement — assets are attached to paragraph indices, and the body filters the same collection at each position:

```
{section.images?.filter((image) => image.afterParagraph === -1)
  .map((image) => <ArticlePhoto image={image} key={image.src} />)}

{section.paragraphs.map((paragraph, paragraphIndex) => (
  <Fragment key={`_${section.id}-${paragraphIndex}`}>
    <p>{renderInlineEmphasis(paragraph)}</p>
    {section.images?.filter((image) => image.afterParagraph === paragraphIndex)
      .map((image) => <ArticlePhoto image={image} key={image.src} />)}
    {section.documents?.filter((d) => d.afterParagraph === paragraphIndex)
      .map((document) => <ArticleDocument document={document} key={document.id} />)}
  </Fragment>
  )})}
```

So: `-1` renders before the first paragraph, `0` after the first, `1` after the second. Documents follow the identical rule. A value pointing past the end of the array simply matches nothing and the asset does not render — no crash, no error. (The importer validates the range, so this only bites hand-edited JSON.)

`<Fragment>` is used with an explicit key because each iteration emits two or more sibling nodes.

Optional blocks, each rendered only when present: `kicker`, `list` (`<ol>` when `listStyle === "ordered"`, otherwise `<ul>`; each item may carry a `bold` label), `references` (external links, all `target="_blank" rel="noreferrer"`), and `quote` (a `<blockquote>`).

Every section is wrapped in `MotionReveal` and carries `id={section.id}` — that id is the scroll target for the table of contents, the anchor in the URL hash, and the key React uses in the list.

ArticleTableOfContents

File `app/components/article-table-of-contents.tsx` · **Kind** Client · 339 lines

Prop	Type
sections	readonly Pick<BlogSection, "id" "title">[]

The most involved component in the codebase. Two presentations (sticky desktop sidebar, mobile disclosure panel) sharing one state, plus scroll-spy that highlights the section you are reading.

State / refs

Name	Purpose
activeId	currently-read section
open	mobile panel open
announcement	text for the live region
tocRef	root, used to find the article container and the active links
toggleRef	mobile button, measured to compute the sticky offset
pendingTargetRef	suppresses scroll-spy during a click-driven smooth scroll
panelId	useId() — collision-free ids for aria-controls

Why not IntersectionObserver? The obvious implementation highlights whichever heading crossed a threshold. That gives wrong answers here: sections vary hugely in length, images shift positions after decode, and a sticky header covers the top of the viewport. This component instead computes, on each frame, **which section occupies the most visible pixels**:

```
const visibleHeight = Math.max(
  0,
  Math.min(sectionBottom, visibleArticleBottom) - Math.max(sectionTop, visibleArticleTop),
);

if (visibleHeight > greatestVisibleHeight) {
  greatestVisibleHeight = visibleHeight;
  currentId = section.id;
}
```

Fallbacks layer on top: if nothing is visible (mid-scroll through a tall image) it picks the last section that starts above the reading line; and at the very bottom of the document it forces the final section, because the last section is often too short to ever win on area.

```
const atDocumentEnd = window.innerHeight + window.scrollY >=
document.documentElement.scrollHeight - 2;
if (atDocumentEnd) currentId = sections.at(-1)?.id ?? currentId;
```

Offsets are measured, not hard-coded. The sticky header's height changes (it condenses), and the mobile toggle may or may not be displayed:


```

const headerBottom = Math.max(0, siteHeader?.getBoundingClientRect().bottom ?? 0);
const compactNavigationVisible = Boolean(
  toggleRef.current && toggleBounds && toggleBounds.height > 0
  && window.getComputedStyle(toggleRef.current).display !== "none",
);
const marker = Math.ceil(compactNavigationVisible && toggleBounds
  ? toggleBounds.bottom + 16
  : headerBottom + 24);
articleRoot?.style.setProperty("--article-sticky-offset", `${marker}px`);

```

The measured value is published back to CSS as `--article-sticky-offset`, so JS measurement and CSS layout cannot disagree.

`getLayoutTop` walks `offsetParent` rather than using `getBoundingClientRect().top + scrollY`, which stays correct inside transformed ancestors — and the page transition applies a transform:

```

function getLayoutTop(element: HTMLElement) {
  let top = 0;
  let current: HTMLElement | null = element;
  while (current) {
    top += current.offsetTop;
    current = current.offsetParent as HTMLElement | null;
  }
  return top;
}

```

The pending-target mechanism. Clicking an entry starts a smooth scroll lasting hundreds of milliseconds, during which scroll-spy would flicker through every intervening section. So a click records the destination:

```

pendingTargetRef.current = { id: section.id, startedAt: performance.now() };
setActiveId(section.id);
target.scrollIntoView({ behavior: reduceMotion ? "auto" : "smooth", block: "start" });
window.history.pushState(null, "", `#${section.id}`);
heading?.focus({ preventScroll: true });

```

While a pending target exists, the spy holds that id and returns early. It is released three ways: arrival (within 32px of the target), the `scrollend` event, or a 1800ms timeout (`PENDING_TARGET_TIMEOUT`). Any user-initiated scroll — wheel, touch, or a scrolling key — cancels it immediately:

```
const cancelPendingNavigation = (event: Event) => {
  if (!pendingTargetRef.current) return;
  if (event instanceof KeyboardEvent
    && ![ "ArrowUp", "ArrowDown", "PageUp", "PageDown", "Home", "End", " " ].includes(event.key))
    return;
  finishPendingNavigation();
};
```

Typing a letter does not cancel; pressing PageDown does.

Everything it subscribes to — `scroll`, `resize`, `hashchange`, `popstate`, `scrollend`, `wheel`, `touchstart`, `keydown`, both `visualViewport` events (mobile URL bar collapse changes the viewport without a window resize), a `ResizeObserver` on the header, the toggle, the article root and every section, and `document.fonts.ready` (a webfont swap reflows everything). All are removed in the cleanup, along with both outstanding animation frames and the observer.

The initial measurement is double-rAF'd — one frame to let layout settle after hydration, one to measure:

```
settleFrame = window.requestAnimationFrame(() => {
  settleFrame = window.requestAnimationFrame(updateActiveSection);
});
```

Keeping the active entry visible. A separate effect scrolls the active link into view inside its own scroll container — with an 8px margin, adjusting only when the link is actually out of bounds, so the list does not jitter:

```
const activeLinks = tocRef.current.querySelectorAll<HTMLAnchorElement>(
  `[data-toc-section="${CSS.escape(activeId)}"]`);
```

`CSS.escape` because section ids come from content and could contain selector-special characters.

`SectionList` is a small local component rendering the `<ol>`; both presentations use it, which is why the desktop and mobile lists cannot drift apart.

Accessibility

- Active entry gets `aria-current="location"` plus an `.sr-only` "Current section" (the checkmark is `aria-hidden`).
- Progress is `<div className="article-toc-progress" aria-hidden>` fed by a `--article-progress` custom property; the same information is available as text ("3 of 7"), so hiding the bar loses nothing.
- Navigating announces through a polite live region: `Moved to section 3: A gunshot at Devil's River`.
- Escape closes the mobile panel and returns focus to the toggle; a pointerdown outside closes it.
- Both `scrollIntoView` calls check `matchMedia("(prefers-reduced-motion: reduce)")` and switch to `"auto"`.

7. Images

ArticlePhoto

File app/components/article-photo.tsx · **Kind** Client

Prop	Type	Default
image	BlogImage	—
hero	boolean	false
priority	boolean	false

A `<figure>` whose image is a button that opens the zoom dialog.

```
<figure className={`article-photo${image.portrait ? " portrait" : ""}${hero ? " hero" : ""}`}>
  <motion.div className="article-photo-motion" whileTap={reduceMotion ? undefined : { scale:
0.997 }} ...>
    <button type="button" className="article-photo-trigger"
      aria-label={`View image: ${image.alt}`}
      aria-haspopup="dialog"
      aria-controls={`focused-image-${image.id}`}
      aria-expanded={viewerOpen}>
      <ResponsiveImage image={image} loading={priority ? "eager" : "lazy"} ... />
      <span className="photo-view"><ZoomIn size={16} aria-hidden="true" /> View</span>
    </button>
  </motion.div>
  <figcaption>{image.caption}</figcaption>
  <FocusedImageDialog image={image} open={viewerOpen} onClose={closeViewer} />
</figure>
```

A real `<button>` rather than a click handler on the image: keyboard focus, Enter/Space activation, and the correct role all come for free. `aria-controls` matches the `id` the dialog renders (`focused-image-${image.id}`).

The press feedback is a 0.3% scale — deliberately below the threshold of conscious notice, enough to register as a physical response. Spring, not duration: `{ type: "spring", stiffness: 520, damping: 34, mass: 0.5 }`.

`closeViewer` is wrapped in `useCallback` because the dialog uses it inside effect dependency arrays.

The dialog is rendered unconditionally with `open={viewerOpen}`; `AnimatePresence` inside it handles mount and exit, which is what makes the close animation possible.

FocusedImageDialog

File app/components/media/focused-image-dialog.tsx · **Kind** Client · 392 lines

Prop	Type
image	BlogImage
open	boolean
onClose	() => void

Full-screen zoom and pan: wheel, pinch, drag, double-click, keyboard.

```
const MINIMUM_SCALE = 1;
const MAXIMUM_SCALE = 5;
const BUTTON_SCALE_STEP = 0.25;
const WHEEL_SCALE_STEP = 0.25;
```

The state/ref duplication. `scale` and `pan` each exist twice — as state (drives render) and as a ref (read by event handlers):

```
useEffect(() => { scaleRef.current = scale; }, [scale]);
useEffect(() => { panRef.current = pan; }, [pan]);
```

Pointer handlers fire between renders and must read the *current* value, not the one captured when the handler was created. Writing both together in `setZoom` keeps them consistent:

```
scaleRef.current = normalizedScale;
panRef.current = constrainedPan;
setScale(normalizedScale);
setPan(constrainedPan);
```

Focal-point zoom — the maths that keeps the pixel under the cursor stationary while scaling:

```
const bounds = stageRef.current.getBoundingClientRect();
const localPoint = {                                     // cursor, relative to stage centre
  x: focalPoint.x - bounds.left - bounds.width / 2,
  y: focalPoint.y - bounds.top - bounds.height / 2,
};
const contentPoint = {                                   // same point in un-transformed image
  x: (localPoint.x - nextPan.x) / currentScale,          space
  y: (localPoint.y - nextPan.y) / currentScale,
};
nextPan = {                                              // pan that puts it back under the
  x: localPoint.x - contentPoint.x * normalizedScale,    cursor
  y: localPoint.y - contentPoint.y * normalizedScale,
};
```

Scale is quantised to quarter steps — `Math.round(nextScale * 4) / 4` — so trackpad pinches land on clean values like 150% instead of 147%.

Pan clamping keeps the image from being dragged off-screen, and collapses to centred whenever the image fits:

```
const maximumX = Math.max(0, (content.offsetWidth * nextScale - stage.clientWidth) / 2);
const maximumY = Math.max(0, (content.offsetHeight * nextScale - stage.clientHeight) / 2);
return { x: clamp(point.x, -maximumX, maximumX), y: clamp(point.y, -maximumY, maximumY) };
```

Pinch tracks live pointers in a `Map` keyed by `pointerId`. On the second pointer down it records the starting distance and scale; each move recomputes the ratio and zooms about the midpoint:

```
const distance = Math.hypot(second.x - first.x, second.y - first.y);
const midpoint = { x: (first.x + second.x) / 2, y: (first.y + second.y) / 2 };
setZoom(pinchStartRef.current.scale * (distance / pinchStartRef.current.distance), midpoint);
```

`setPointerCapture` means a drag continues correctly even when the pointer leaves the element, and `onLostPointerCapture` is wired to the same cleanup as `pointerup/pointercancel` so no gesture can leak.

Input map

Input	Effect
wheel	zoom ± 0.25 about the cursor (ignored with Ctrl/Cmd — that is browser page zoom)
double-click	2× at the click point, or reset if already zoomed
drag	pan (only above minimum scale, only primary button)
pinch	zoom about the midpoint
+ / = / -	zoom step
0	reset
arrows	pan 36px, or 80px with Shift
Escape	close
Tab	trapped

Zoom keys bail out if a modifier is held, so browser shortcuts still work.

Auto-hiding controls. The dock appears on mouse enter or focus and hides 650ms after leaving — but never while zoomed, and never while focus is inside it:

```
const scheduleControlsHide = () => {
  if (scale > MINIMUM_SCALE) return;
  hideControlsTimerRef.current = setTimeout(() => {
    if (!controlsShellRef.current?.contains(document.activeElement)) setControlsOpen(false);
  }, 650);
};
```

The timer is also cleared on unmount by a dedicated effect.

Portal. The dialog renders into `document.body` via `createPortal<`